Inference scaling
A hard worksheet for today

Calculations, counterexamples, and architecture decisions

This is an original problem set built around the scaling book and Reiner Pope's two Dwarkesh blackboard lessons. It assumes you have read the book. Problems and numbers are newly constructed; they are not copied exercises or Reiner's official answers.

Time budget

Questions 1-10:280 minutes of working time (about 4 hours 40 minutes), plus breaks. Questions 11-12 add 50 minutes. For a shorter first session, do Q1-Q4 (110 minutes), then resume Q5-Q10. Take the estimates as pacing targets; an unfamiliar derivation can take longer.

How to work

Start each problem with a bottleneck prediction. Use paper or a notebook for derivations, then a calculator or a short script to check arithmetic. First pass: no book, transcript, answer key, or AI-generated solution. Mark exactly where you become stuck; on a second pass, consult the indicated source segment.

What earns credit

A complete answer states the model, tracks units, derives the limiting term, checks memory and workload feasibility, gives a quantitative result, and names at least one condition under which the conclusion would change. A roofline lower bound is not a measured latency or a queueing guarantee.

Coverage

Problem	Main challenge
1-3	Dense and MoE rooflines; expert occupancy; tensor-parallel communication
4-6	Cyclic decode pipelines; inverse pricing; lifetime training/serving optimization
7-8	KV transfer tails; speculative verification under load
9-10	Area and energy tradeoffs; array shape and scratchpad tiling
11-12	Training sharding; benchmark attribution and SLO goodput

Shared reference sheet

Unless a question explicitly overrides a value, use these hypothetical effective rates. They describe a toy system, not a named commercial GPU.

Accelerator A	Value
Effective BF16 compute, F	600 TFLOP/s =600e12 FLOP/s
Effective HBM bandwidth, W	3 TB/s =3e12 bytes/s
HBM capacity, H	80 GB =80e9 bytes
Fixed reserve, U	8 GB/GPU; unavailable to weights and KV
Rental cost	US $2.40/GPU-hour

Dense model D	Value
Parameters, P	30 billion; excludes bookkeeping-only operations
Layers, L	48
Query heads, h_q	48
KV heads, h_kv	8 (grouped-query attention)
Head dimension, d	128
Weight and KV precision	BF16,2 bytes/value
Query/hidden width	h_q*d=6,144

Units and modeling rules

GB, TB, TFLOP and PFLOP use decimal powers of 10. KiB uses 1,024 bytes. One multiply-accumulate is two FLOPs. B is the number of requests emitting one token each in a decode step; T is their cached context length. G is global resident request population; b is local pipeline microbatch size.

Rates labeled effective are already efficiency-adjusted: do not apply another guessed utilization factor. When a question grants perfect overlap, use max only for the granted resources. Otherwise preserve serial terms and dependencies. Ignore only the terms a question explicitly excludes.

Tensor shorthand

At one layer, Q has shape[B,h_q,d] for decode, while cached K and V each have shape[B,T,h_kv,d]. Each KV head is shared by h_q/h_kv query heads. Attention still computes over all query heads; KV storage uses only KV heads.

Source tags

B1-B3 identify the scaling book; R1-R2 identify the two Reiner/Dwarkesh lessons; P1-P2 identify research papers. The source map on the final worksheet page provides links and timestamps. You do not need to rewatch them before your first attempt.

Q01 Find the bottleneck before finding the batch

10 points | 30 minutes | Source: B1, B2; R1 at 00:00:00

Use accelerator A and dense model D from the reference sheet. Each active request has T = 4,096 cached tokens; B requests generate one token each in a decode step. Read every weight once per step and every request's cached K/V once. Ignore KV writes, padding, and activation traffic.

For projections and MLPs, count 2PB FLOPs per step. For QK and AV attention together, count 4LBTh_qd FLOPs. For this toy model, treat compute and HBM traffic as perfectly overlapped across the whole step; the step time equals their larger time. This is an optimistic model, not a promised measured runtime.

(a) [2 points] Derive KV bytes per cached token per request, kappa. Derive t_compute(B,T), t_memory(B,T), and the memory-capacity constraint from tensor shapes.

(b) [2 points] At T = 4,096, report compute time, memory time, throughput, and bottleneck for B = 1, 8, and the largest feasible integer B. Sketch throughput against B.

(c) [3 points] A colleague claims that a sufficiently large batch always makes this model compute-bound. Derive a context-length threshold above which no finite batch can do so, even with unlimited HBM capacity. Explain the equality case.

(d) [3 points] Choose the throughput-maximizing feasible batch under a 25 ms inter-token service-time limit, excluding queueing. Compute dollars per million output tokens. Explain why this is not a p95 user-latency guarantee.

Derivation / sketch / decision

Q02 MoE: active parameters are not active weight traffic

10 points | 30 minutes | Source: B2, P1; R1 at 00:32:09

A hypothetical MoE has 64 experts, each with 8 GB of BF16 weights, plus 32 GB of shared weights. Each token chooses exactly two distinct experts uniformly; choices are independent across tokens. Sixteen accelerators provide pooled effective compute $F_{pool}=9.6$ PFLOP/s and pooled HBM bandwidth $W_{pool}=48$ TB/s.

Ignore attention, KV, network, and placement overhead in parts (a)-(c). Every expert touched anywhere in the batch is read once; shared weights are read once. Assume compute is 2 FLOPs per active parameter per token and the ideal step time is max(compute time, expected byte-traffic time). This uses expected traffic, not an exact tail-latency distribution.

(a) [3 points] Derive the expected number $U(B)$ of distinct experts touched by B tokens. Derive expected weight traffic and compute FLOPs. Explain why neither 48 GB nor 544 GB is the right traffic estimate for every batch size.

(b) [2 points] Evaluate U, compute time, and memory time at $B = 1$, 8, 32, 128, and 512. Identify the limiting resource at each point.

(c) [2 points] In the saturated-expert limit, estimate the critical batch. Express it using the hardware FLOPs/byte ratio and the ratio of total to per-token active parameters.

(d) [3 points] Now place four experts on each accelerator. One hotspot expert is selected with probability 1/4 per token; the remaining expected 7/4 selections are uniform across the other 63 experts. Shared compute is negligible here. Derive the hotspot accelerator's expected load relative to the ideal mean accelerator load. What must change in the compute roofline?

Derivation / sketch / decision

Q03 More tensor parallelism can make you slower

10 points | 25 minutes | Source: B1, B2, P1; R1 at 00:32:09

Use model D at B=8 and T=4,096 on n in {1,2,4,8,16} accelerators A. Weight parameters and compute divide evenly over n. KV is evenly sharded only through n=8; at n=16 each KV shard is replicated twice, so each rank holds total KV bytes / min(n,8). Ignore other divisibility constraints.

Each of the 48 layers performs two SERIAL ring all-reduces. For one collective per rank, t_AR = alpha(n-1) + [2(n-1)/n] S/W_link, where alpha=5 microseconds, W_link=150 GB/s, and S=B*6,144*2 bytes. Compute and HBM overlap perfectly within the local work; all collectives are added serially after that local roofline time.

(a) [3 points] Derive total step time and per-rank capacity. Tabulate local roofline, all-reduce time, and total time for every n. Check feasibility before choosing n.

(b) [2 points] Choose n for the lowest inter-token service time. Explain why n=16 can lose despite more compute and smaller weight shards.

(c) [2 points] For the relaxed range n<=8, treat n as continuous. Derive the stationary point of the total time and compare it with the allowed integer choices.

(d) [3 points] With eight GPUs available and enough independent requests, compare one n=8 replica serving B=8 against four n=2 replicas each serving B=8. Compute aggregate output throughput and per-request step time. State which objective each layout serves better.

Derivation / sketch / decision

Q04 A decode pipeline is a closed loop

10 points | 25 minutes | Source: B2, P1; R1 at 00:47:12 and 01:03:37

Four pipeline stages serve M fixed microbatches, each with b requests. Every microbatch returns to stage 1 only after its token passes through stage 4. Stages are deterministic; feedback is instantaneous; there is no network or sampling delay. Each stage's service time is $t_{stage}(b)=\max(3,0.02b)$ ms. A stage serves one microbatch at a time.

On every GPU, weights occupy 30 GB, reserve occupies 10 GB, and each globally resident request contributes 0.05 GB of KV on that GPU. Capacity is 80 GB/GPU. Require steady-state inter-token time $<=40$ ms. Optimize among positive integers M,b; use a periodic work-conserving cyclic schedule.

(a) [3 points] Draw a time-slot schedule for M=1,3,4 at b=150. Derive steady-state token throughput and per-request inter-token time as functions of M and b. Distinguish warm-up bubbles from persistent idle time.

(b) [3 points] Find the maximum attainable throughput subject to capacity and latency. Among equally fast layouts, choose the smallest global population G=Mb, then the smallest inter-token time. Report M,b,G, throughput, and latency.

(c) [2 points] A colleague applies finite-pass pipeline efficiency M/(M+3) to steady-state decode. Explain the mismatch and give the correct utilization here.

(d) [2 points] At fixed b=150, change the four stage times to 3,3,6,3 ms. Give a lower bound on the microbatch population needed to saturate the bottleneck, exhibit a periodic schedule that attains it, and compute steady-state inter-token time.

Derivation / sketch / decision

Q05 What API prices do and do not identify

10 points | 25 minutes | Source: B2; R1 at 01:33:02

A fictional provider charges p(T)=2.40*(1+T/40,000) dollars per million output tokens for a fixed context T. Assume this is exactly its serving cost: no margin, discount, cross-subsidy, or cached-prefix benefit. Its serving pool costs r=$0.006/s and delivers effective pooled HBM bandwidth W=48 TB/s.

Assume memory-bound decode, constant batch B, m=480 GB of weight reads per step, and kappa bytes of KV per cached token per request. Then a step reads m+B*kappa*T bytes and emits B output tokens. Ignore compute, network, and writes.

(a) [3 points] Derive p(T) from the serving model. Infer B and kappa using the supplied m,r,W. Keep dollars per million tokens separate from dollars per token.

(b) [2 points] If kappa halves and everything else stays fixed, what happens to the price intercept, slope, and normalized context scale? Repeat if only m halves.

(c) [3 points] Relax the assumption that m is known: exhibit two different (m,B,kappa) tuples with exactly the same price curve. Then allow an unknown multiplicative price markup; show a second ambiguity affecting inferred KV bytes.

(d) [2 points] An analyst claims this curve proves a provider uses a specific attention architecture and batch size. State the strongest defensible conclusion under the toy assumptions, and identify at least three assumptions that would break it in production.

Derivation / sketch / decision

Q06 Train longer to serve a smaller model

10 points | 35 minutes | Source: B3; R1 at 01:18:59

Use a dimensionless toy scaling law: excess loss ell(N,D)=N^(-1/2)+D^(-1/2). Here N>1 is model size and D is training-token count in normalized units. Require ell<=1. Total lifetime compute cost is C(N,D;R)=6ND+2NR, where R>=0 is lifetime inference-token demand in the same token units as D.

N,D are positive continuous variables. This objective ignores hardware efficiency, memory costs, latency, and changing inference quality. Do not import the interview's empirical overtraining multiples into this artificial scaling law.

(a) [2 points] Show that an optimum lies on ell=1; eliminate D and state the valid domain for N.

(b) [3 points] Solve the R=0 optimum. For general R, substitute x=N^(-1/2), derive a scalar stationarity equation, and argue that the relevant root is unique.

(c) [2 points] At R=30, solve numerically for N,D,C. Compare with keeping the R=0 model and training-token count fixed.

(d) [3 points] Derive the large-R asymptotic scaling of N-1 and D. Explain what this teaches about overtraining for lifetime inference demand, and which real-world terms could reverse the recommendation.

Derivation / sketch / decision

Q07 Disaggregation: capacity is not a latency guarantee

10 points | 30 minutes | Source: B2; original capacity and tail-latency extension

Use model D with T=32,768; transfer its full uncompressed KV after prefill. A prefill GPU takes 120 ms/request. A decode GPU emits 3,200 output tokens/s at its stipulated operating point. Requests arrive at 12/s and emit 500 output tokens each. Each independently provisioned fabric link carries 25 GB/s. Transfers add 2 ms of fixed overhead.

Parts (a)-(b) concern ideal capacity and queue-free TTFT. In (c), allow all links to stripe each individual transfer perfectly. In (d), return to UNSTRIPED transfers: 75% of requests find an exact reusable prefix covering 75% of their context already on the chosen decode node, so only the remaining 25% of KV must move; misses transfer all KV. Prefill still takes 120 ms on either path. Ignore compute/cache effects beyond these stipulations.

(a) [2 points] Compute full KV bytes/request. Find the minimum integer counts of prefill GPUs, decode GPUs, and links needed for offered-load stability in the ideal fluid model. Require strictly more capacity than offered load.

(b) [2 points] With unstriped transfers and no cache reuse, compute queue-free TTFT. Does meeting all aggregate capacity requirements prove a 250 ms p95 TTFT target?

(c) [2 points] If all minimum-count links stripe every request perfectly, compute queue-free TTFT and aggregate bandwidth utilization. Explain why even this is only a necessary starting point for a p95 claim.

(d) [4 points] Under the stipulated hit/miss mixture, compute average transferred bytes, minimum link count, hit TTFT, miss TTFT, and the queue-free 95th percentile. Decide whether average bandwidth savings rescue the p95 target. Identify one measurement needed to make a production recommendation.

Derivation / sketch / decision

Q08 Speculation that helps at low load and hurts at high load

10 points | 25 minutes | Source: B2, P2; original speculative-serving model

Baseline decode step time is $t_0(B)=\max(8+0.04B,0.10B)$ ms. A speculative cycle drafts $K=4$ tokens per request, then verifies them and samples one correction/bonus token. Each successive draft token has conditional acceptance probability $q=0.75$, independent of previous acceptance events; verification stops logically at the first rejection.

Draft time is $d(B,K)=1.5+0.02BK$ ms. Verification time is $v(B,K)=\max(8+0.04B,0.10B(K+1))$ ms. Draft and verification are serial. Compare the same batch B, output distribution, and hardware allocation. Ignore memory-capacity changes; these are stipulated timing laws, not claims about a particular engine.

(a) [3 points] Derive expected committed output tokens per request per cycle, including the correction/bonus token. Derive throughput speedup and effective per-token service time.

(b) [2 points] Evaluate speedup at $B=1,16,64,128$. Why can the acceptance probability be unchanged while the optimization becomes harmful?

(c) [3 points] Solve the piecewise timing inequalities to find every integer batch size at which $K=4$ improves throughput. Check every roofline branch, rather than extrapolating one regime.

(d) [2 points] For stochastic sampling, explain why accepting draft tokens merely when their argmax matches the target does not preserve the target distribution. State the acceptance rule and the distribution used to sample a token after rejection.

Derivation / sketch / decision

Q09 Why four-bit arithmetic does not imply a fourfold chip win

10 points | 25 minutes | Source: R2 at 00:00:00, 00:16:31, and 00:39:11; B1

A fictional chip has area budget 60,000 units for identical MAC lanes. Each lane with p-bit multiply inputs and a fixed 32-bit accumulator uses A(p,s)=p^2+4p+64+20(s-1) area units, where s in {1,2,4} is pipeline depth. Multiplication logic contributes p^2; routing contributes 4p; accumulation contributes 64; extra registers contribute 20(s-1).

Critical logic delay is 2/s ns and register overhead is 0.2 ns. Every lane initiates one MAC per clock after fill; one MAC counts as two FLOPs. Floor the lane count. Ignore power limits in parts (a)-(c). A workload has arithmetic intensity 4 FLOPs/byte at p=8, doubling at p=4 because the stipulated total byte traffic halves. HBM bandwidth is 100 GB/s. For part (d), energy per MAC is e(p)=0.5p^2+6.4+48 pJ; the last term is fixed data-movement energy.

(a) [3 points] For all six (p,s) combinations with p in {8,4}, compute lane count, clock rate, and peak FLOPs/s. Identify the maximum-peak configuration for each precision.

(b) [2 points] Compare the best p=4 and p=8 peak rates. Reconcile the observed ratio with the quadratic multiplier-area term. Explain why faster clocks alone are not a throughput metric.

(c) [3 points] Apply the workload roofline to both best configurations. Find realized throughput and speedup. Does the peak ratio bound the actual speedup when precision also changes byte traffic?

(d) [2 points] Calculate energy/MAC and ideal MACs/joule at both precisions. Explain why keeping accumulation and movement fixed changes the precision benefit; identify what evidence is still needed before using four-bit arithmetic for a real model.

Derivation / sketch / decision

Q10 Systolic arrays: shape, fill time, and scratchpad capacity

10 points | 30 minutes | Source: R2 at 00:26:10 and 01:03:25; B1, B3

Multiply X of shape [16,1,024] by W of shape [1,024,256]. Inputs are BF16; accumulators/output are FP32. A 128x128 systolic array runs at 1 GHz. For an independent tile with reduction length K, time is K+R+C-2 cycles on an RxC array, including fill/drain. Partial row/column tiles still pay the physical array's full fill/drain cost. Output column tiles execute serially.

Part (b) may compare a 32x128 array at the same clock and may stack eight independent X matrices into 128 rows. For part (c), use a 256 KiB scratchpad (1 KiB=1,024 bytes), one output tile with m_t=16,n_t=128, double-buffered BF16 input/weight tiles, and one live FP32 accumulator tile. K chunks must be multiples of 32 except the final chunk. Every K chunk independently pays fill/drain in this toy schedule. No HBM/compute overlap is assumed.

(a) [2 points] Compute useful MACs, total cycles, and utilization of the 128x128 array. Utilization means useful MACs divided by physical peak MACs over all elapsed cycles.

(b) [3 points] Repeat on the 32x128 array and for eight stacked requests on the 128x128 array. If eight requests arrive 20 microseconds apart and dispatch waits for all eight, compute the first request's added batching wait and average batching wait. Separate kernel efficiency from user latency.

(c) [3 points] Derive scratchpad occupancy as a function of K_t. Find the largest legal K_t, a partition of K=1,024, and the new total array-cycle count. With no reuse across output-column tiles, count total HBM operand reads and output writes.

(d) [2 points] Compare a cache-managed schedule with an explicit scratchpad schedule: what has to be known or proved about lifetimes, layout, and reuse? Explain why double buffering by itself does not remove the independently stipulated array fill/drain costs.

Derivation / sketch / decision

Q11 Stretch: training memory savings can expose communication

10 points | 25 minutes | Source: B1, B3; original full-sharding training model

Train a 7-billion-parameter dense model on eight GPUs. Each GPU has 80 GB HBM, a 12 GB fixed reserve, F=600 TFLOP/s effective compute, and W_link=100 GB/s effective collective bandwidth. Persistent state per parameter is: BF16 weight 2 bytes, BF16 gradient 2 bytes, FP32 master weight 4 bytes, and FP32 Adam moments 8 bytes. Activations occupy 24 GB/GPU at the stipulated batch.

Full parameter sharding divides all persistent state evenly over eight GPUs and adds 2 GB/GPU of temporary live full-layer buffers. There are 32 layers. Per step, gather BF16 weights twice and reduce-scatter BF16 gradients once; aggregate communication bytes per rank are each full-model message size times 7/8. Each of the 96 collectives adds 20 microseconds startup. There are S=8,192 global training tokens per step; compute is 6PS FLOPs split evenly over GPUs. For part (c), hold all efficiencies fixed.

(a) [3 points] Check whether replicated-state data parallelism fits. Check full sharding, including reserve, activations, and temporary buffers. Explain which memory category sharding does not automatically reduce.

(b) [4 points] Derive per-rank compute and communication times. Report a no-overlap step-time model and the optimistic perfect-overlap lower bound. Identify the bottleneck.

(c) [3 points] Find the global S where compute and communication times match. Does that batch necessarily fit if activation memory grows linearly with S? Derive the maximum S allowed by the given memory budget and compare.

Derivation / sketch / decision

Q12 Stretch: cross-examine an inference speedup claim

10 points | 25 minutes | Source: B3, P2; original benchmark critique

Two serving reports use eight identical GPUs at a total cost of $24/hour. Every completed request emits exactly 500 output tokens. Report A:12 completed requests/s, p95 TTFT 180 ms, p95 per-request mean inter-token time 35 ms, prefix-hit rate 90%. Report B:16 requests/s, p95 TTFT 700 ms, p95 per-request mean inter-token time 18 ms, prefix-hit rate 20%. Prompt-length and arrival distributions were not reported.

The product requires at least 95% of completed requests to satisfy BOTH TTFT<=250 ms and per-request mean inter-token time<=25 ms. Separately, a developer reports an isolated attention kernel improved from 5 ms to 2 ms; an otherwise unchanged serial model step originally spent 15 ms outside that kernel. Interpret p95 as an exact population quantile for the reasoning exercise.

(a) [3 points] Compute raw output throughput and infrastructure cost per million output tokens for A and B. Determine what the p95 statistics establish about the joint product target. Can you calculate exact goodput from these summaries?

(b) [3 points] Specify a controlled experiment that can attribute a serving improvement to the implementation: workload, cache state, demand, quality/correctness, resource accounting, and repeated measurement. Explain why closed-loop client concurrency alone is insufficient to characterize overload.

(c) [2 points] Compute the largest end-to-end speedup explained by the isolated kernel improvement under the given serial model. Express the calculation using Amdahl's law.

(d) [2 points] Write a four-sentence review of the claim: one verified result, one limitation, one corrected quantitative conclusion, and one decisive next measurement. Avoid dismissing a useful local optimization merely because the overall claim is overstated.

Derivation / sketch / decision

Source map

The problems are original applications and extensions. Source tags indicate conceptual starting points, not copied question text or endorsement by the speakers. Numerical hardware, prices, and workloads are deliberately hypothetical.

B1 / How to Scale Your Model: rooflines

https://jax-ml.github.io/scaling-book/roofline/

B2 / How to Scale Your Model: transformer inference

https://jax-ml.github.io/scaling-book/inference/

B3 / How to Scale Your Model: full book and chapter index

https://jax-ml.github.io/scaling-book/

R1 / Reiner Pope / Dwarkesh: training and serving math (April 29, 2026)

https://www.dwarkesh.com/p/reiner-pope

R2 / Reiner Pope / Dwarkesh: chip design (May 22, 2026)

https://www.dwarkesh.com/p/reiner-pope-2

P1 / Pope et al.: Efficiently Scaling Transformer Inference

https://arxiv.org/abs/2211.05102

P2 / Liu et al.: Speculative Decoding: Performance or Illusion?

https://arxiv.org/abs/2601.11580

Reiner lesson links and useful segments

Training/serving lesson on YouTube: batching 00:00:00; MoE 00:32:09; pipeline 00:47:12 and 01:03:37; lifetime demand 01:18:59; inverse pricing 01:33:02.

Chip-design lesson on YouTube: MACs 00:00:00; mux/data movement 00:16:31; systolic arrays 00:26:10; pipeline registers 00:39:11; cache/scratchpad 01:03:25.

After solving

For three answers, name a real implementation detail excluded by the model and predict how it changes your conclusion. Then check the separate key. Distinguish an incorrect derivation from a correct derivation under a different, clearly stated assumption.